TOPIC

MICROSERVICE DESIGN PRINCIPLES

Ashish Sahu

Tech Career Coach

MONITOR CONSTANTLY

- **Implement centralized logging (ELK, Loki) –** Aggregate logs from all services for better troubleshooting.
- **Use distributed tracing (Jaeger, Zipkin) for request tracking** – Track requests across multiple services to identify performance bottlenecks.
- **Set up metrics & alerts (Prometheus, Grafana) –** Monitor key performance indicators like CPU usage, latency, and error rates.
- **Monitor latency, errors, traffic, and saturation (SRE principles)** – Follow Google's SRE (Site Reliability Engineering) principles for better observability.
- **Perform health checks & synthetic monitoring –** Run periodic health checks and simulated requests to catch failures proactively.

BUILD AROUND BUSINESS CAPABILITIES

- **Organize services by business domains (e.g., Order, Payment, Inventory) –** Ensure microservices align with business workflows, not just technology layers.
- **Avoid technical partitioning (e.g., "AuthService," "DatabaseService") –** Services should not be defined based on technical components but on business needs.
- **Teams should align with business functions (cross-functional teams) –** Development teams should be structured around the services they own.
- **Encourages domain ownership and better decision-making –** Each team should have full control over its domain to make efficient decisions.
- **Makes the system more adaptable to business changes –** Service boundaries should be flexible to accommodate evolving business requirements.

MANAGE TRAFFIC (MAGIC TRAFFIC)

- **Use load balancers (Nginx, HAProxy) for even distribution –** Balance incoming requests across multiple service instances.
- **Implement rate limiting & throttling (to prevent abuse) –** Prevent API overuse and ensure fair resource distribution.
- **Apply blue-green/canary deployments for smooth rollouts** – Deploy updates gradually to minimize risks.
- **Use feature flags for controlled feature releases –** Roll out features selectively to different user groups.
- **Optimize caching strategies (CDN, Redis) to reduce load** – Reduce backend load by caching frequently requested data.

ENABLE INTER-SERVICE COMMUNICATION

- **Prefer asynchronous (events/messages) over synchronous calls (REST/gRPC) –** Reduce coupling and improve resilience with event-driven communication.
- **Use message brokers (Kafka, RabbitMQ) for event-driven architecture –** Decouple services by using messaging systems.
- **Implement API gateways for routing and aggregation –** Centralize API requests, authentication, and rate limiting.
- **Apply service mesh (Istio, Linkerd) for observability & resilience –** Manage communication security, retries, and tracing between services.
- **Document APIs using OpenAPI/Swagger –** Ensure all services provide clear, standardized API documentation.

GEAR UP PROCESS AUTOMATION

- **Automate CI/CD pipelines for fast, reliable deployments –** Use tools like Jenkins, GitHub Actions, or GitLab CI/CD.
- **Use Infrastructure as Code (IaC) (e.g., Terraform, Ansible) –** Manage infrastructure configuration using code for reproducibility.
- **Automate testing (unit, integration, contract, chaos testing) –** Implement automated tests to detect issues early in the development cycle.
- **Implement auto-scaling based on demand –** Ensure services can scale dynamically based on workload.
- **Use container orchestration (Kubernetes, Docker Swarm) –** Manage and scale microservices efficiently with containerized deployments.

DECENTRALIZE DATA

- **Each service should own its database (no shared DB) –** Avoid a central database, as it leads to strong coupling between services.
- **Prefer database-per-service over a monolithic database –** Each service should have its dedicated storage to maintain autonomy.
- **Use eventual consistency where strong consistency isn’t needed –** Instead of distributed transactions, allow services to sync asynchronously.
- **Implement CQRS (Command Query Responsibility Segregation) if needed –** Separate read and write operations to improve scalability.
- **Avoid distributed transactions (use Saga pattern instead) –** Use the Saga pattern to manage complex, multi-step workflows without strong coupling.

ENSURE HIGH COHESION & LOW COUPLING

- **Group related functionalities within a single service** – Each service should encapsulate a well-defined business function. Avoid mixing unrelated functionalities.
- **Minimize dependencies between services** – Ensure services interact via well-defined APIs or events, rather than tightly coupling their logic.
- **Avoid shared databases between services** – Each service should manage its own data store to prevent tight dependencies and ensure autonomy.
- **Design services to be independently deployable** – Each microservice should be deployable without affecting others, enabling continuous delivery.
- **Use well-defined interfaces for communication** – Establish clear API contracts (REST, gRPC, GraphQL) and avoid direct database access between services.

DESIGN FOR FAILURE

- **Assume services will fail—implement resilience patterns –** Plan for unexpected failures and design fallback strategies.
- **Use circuit breakers (e.g., Hystrix, Resilience4j) to prevent cascading failures –** Temporarily block failing services to prevent further damage.
- **Apply retries with exponential backoff for transient errors –** Retry mechanisms should gradually increase wait times between attempts.
- **Implement timeouts to avoid indefinite waits –** Define limits on how long a service should wait before timing out.
- **Use bulkheads to isolate failures in one service from others –** Prevent a failure in one service from affecting the entire system by isolating resource pools.

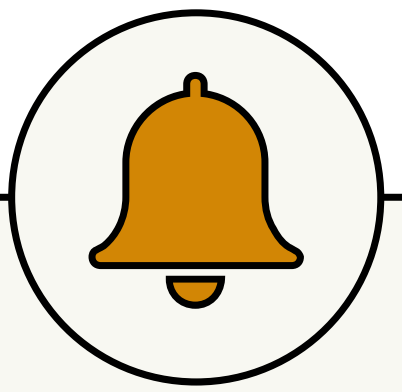
ADHERE TO THE SINGLE RESPONSIBILITY PRINCIPLE

- **Each service should handle one specific business function –** A service should be designed around a single responsibility, such as "Payment Processing" or "User Authentication."
- **Avoid mixing unrelated logic in a single service –** Keep separate concerns apart to ensure code clarity and maintainability.
- **Changes to one service should not require changes in others –** A microservice should evolve independently without impacting other services.
- **Improves maintainability and scalability –** Smaller, focused services are easier to debug, scale, and maintain.
- **Makes debugging and testing easier –** Services with a clear scope are simpler to test and troubleshoot.

DEFINE THE SCOPE PROPERLY

- **Follow the "Bounded Context" concept from DDD –** Clearly define service boundaries based on business domains rather than technical aspects.
- **Avoid creating too large (monolithic) or too small (nanoservices) services –** Overly large services defeat the purpose of microservices, while nanoservices create excessive complexity.
- **Align service boundaries with business capabilities –** Structure services around business processes like "Order Management" rather than technical concerns.
- **Ensure each service has a clear, distinct purpose –** Services should have single, well-defined objectives, avoiding overlaps with other services.
- **Re-evaluate and adjust scope as business needs evolve –** As businesses grow, service boundaries should be revisited to ensure they remain optimal.

FOLLOW



Ashish Sahu

Tech Career Coach